

PASTA Threat Modeling Framework – Sneaker Company App

Scenario

You're part of the growing security team at a company for sneaker enthusiasts and collectors. The business is preparing to launch a mobile app that makes it easy for their customers to buy and sell shoes. You are performing a threat model of the application using the PASTA framework. You will go through each of the seven stages of the framework to identify security requirements for the new sneaker company app.

Stages	Sneaker company
I. Define business and security objectives	• Users can create member profiles internally or by connecting external accounts (e.g., Google, Apple). • The app must process financial transactions, including credit/debit card and peer-to-peer payments. • The app should comply with PCI-DSS standards to ensure payment data is secure.
II. Define the technical scope	- Application Programming Interface (API) - Public Key Infrastructure (PKI) - SHA-256 - SQL APIs are prioritized because they are the main communication bridge between the app, users, and third-party services. They often handle sensitive operations like authentication, transactions, and data exchange. Because they're exposed to the internet, APIs introduce a large attack surface and require strong security protections.
III. Decompose application	1. User Device → API Gateway: Sends login credentials or connects third-party accounts 2. API Gateway → Auth Server: Verifies user identity using PKI 3. User Device → App Server: Submits sneaker listings, purchases, or messages 4. App Server → Database (SQL): Stores or retrieves listing data 5. App Server → Payment Processor: Processes transactions securely 6. Database → API Gateway: Returns requested sneaker data, user profiles, etc.
IV. Threat analysis	• Injection: Attackers may try SQL injection attacks if inputs are not sanitized. • Session Hijacking: Attackers may hijack user sessions to impersonate users or steal information.

	● Internal Threats: ● Developers or employees with excessive database privileges. ● External Threats: ● Hackers attempting to breach authentication mechanisms or inject malicious code via exposed APIs.
V. Vulnerability analysis	● Lack of Prepared Statements: Leaves the database vulnerable to SQL injection. ● Broken API Token Management: If tokens are not rotated or securely stored, attackers could reuse them.
VI. Attack modeling	1. Exploit a Vulnerability in the App: - Look for input fields not using prepared SQL statements - Perform SQL Injection to access or modify payment data 2. Hijack a User Session: - Use Cross-Site Scripting (XSS) to steal a session cookie - Replay the session to act as the user without logging in 3. Abuse API Authentication: - Find an exposed or weak API token - Use the token to bypass login and view user information 4. Launch a Phishing Attack: - Create a fake login page that looks like the real app - Trick users into entering their username, password, or card info
VII. Risk analysis and impact	Security Controls to Reduce Risk: ● SHA-256: To hash and secure sensitive data such as passwords. ● Incident Response Procedures: To detect and react to breaches quickly. ● Password Policy: To enforce strong passwords and prevent brute force attacks. ● Principle of Least Privilege: To ensure users and employees only access necessary resources.